გაკვეთილი 8 Mod 7 (7.2)

გაკვეთილი 4 — Git Pull და გუნდური მუშაობა GitHub-ზე

2. თეორიული ნაწილი

2.1. რატომ გვჭირდება Git Pull?

წარმოიდგინეთ, რომ ორი მოსწავლე — ნიკა და მარიამი — ერთად ქმნის სკოლის ვებსაიტს.

ორივემ GitHub-დან გადმოიტანა ერთნაირი პროექტი:

```
school-website/
    index.html
    style.css
```

ნიკამ პროექტში დაამატა ახალი გვერდი `about.html` და ატვირთა GitHub-ზე.

ახლა GitHub-ზე პროექტი ასე გამოიყურება:

```
school-website/
    index.html
    style.css
    about.html
```

მაგრამ მარიამის კომპიუტერში კვლავ ძველი ვერსიაა:

```
school-website/
    index.html
    style.css
```

როგორ მიიღოს მარიამმა ნიკას მიერ დამატებული ფაილი ისე, რომ პროექტის თავიდან გადმოტანა არ მოუწიოს?

ამისთვის გამოიყენება `git pull`.

2.2. რა არის Git Pull?

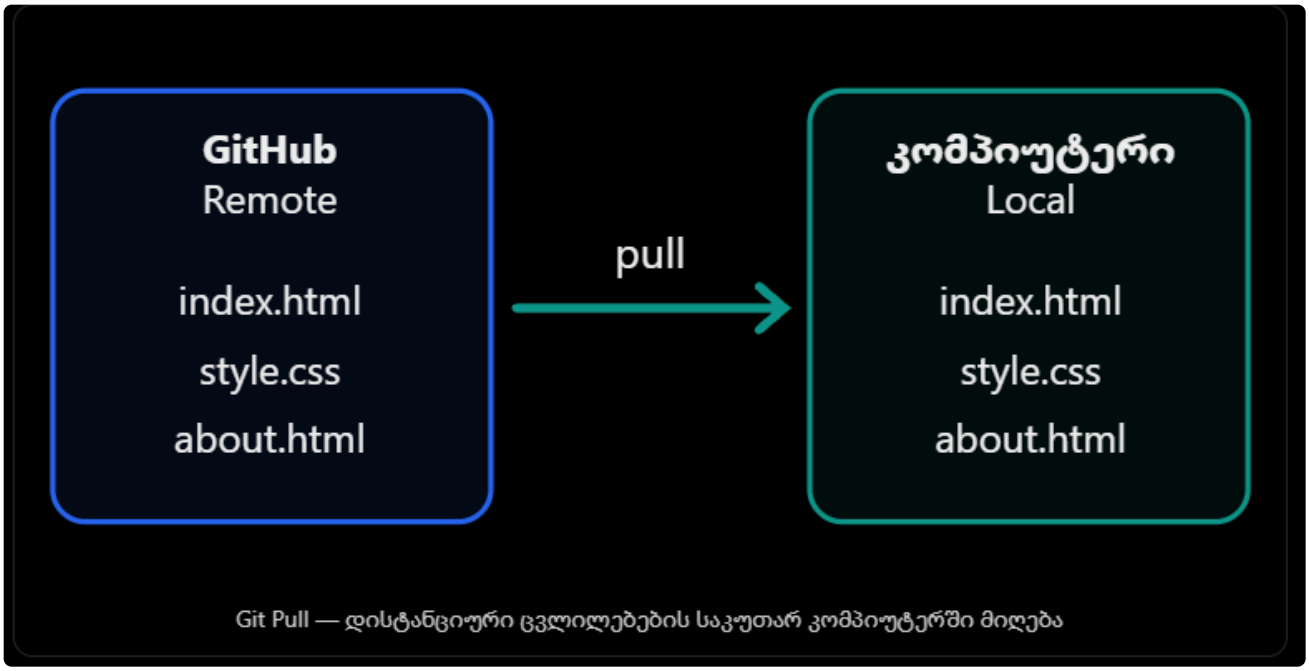
`git pull` არის Git-ის ბრძანება, რომლის საშუალებითაც დისტანციური რეპოზიტორიიდან ვიღებთ ახალ ცვლილებებს და ვაერთიანებთ მათ მიმდინარე ლოკალურ Branch-ში.

ბრძანება:

```
git pull origin main
```

აქ:

- `git pull` — ცვლილებების მიღება და გაერთიანება;
- `origin` — დისტანციური რეპოზიტორიის სახელი;
- `main` — დისტანციური Branch, საიდანაც ცვლილებები უნდა მივიღოთ.



Git Pull — დისტანციური ცვლილებების საკუთარ კომპიუტერში მიღება

თუ მარიამი თავის პროექტში შეასრულებს:

```
git checkout main
git pull origin main
```

მის კომპიუტერშიც გამოჩნდება `about.html`.

ამრიგად, მას უკვე პროექტის განახლებული ვერსია ექნება.

2.3. რა განსხვავებაა Git Push-სა და Git Pull-ს შორის?

ეს ორი ბრძანება ერთმანეთისგან უნდა განვასხვაოთ.

Git Push	Git Pull
ცვლილებებს აგზავნის GitHub-ზე	ცვლილებებს იღებს GitHub-იდან
ლოკალურ commit-ებს ატვირთავს	დისტანციურ ცვლილებებს აერთიანებს

Git Push	Git Pull
ვიყენებთ სამუშაოს ატვირთვისას	ვიყენებთ პროექტის განახლებისას

მაგალითად:

```
git add .
git commit -m "Added new page"
git push origin main
```

ამ შემთხვევაში ჩვენი ცვლილებები GitHub-ზე აიტვირთება.

ხოლო:

```
git pull origin main
```

GitHub-ზე არსებული ახალი ცვლილებების მიღებასა და გაერთიანებას ცდილობს.

დაიმახსოვრეთ:

`Push` — ჩვენი ცვლილებები მიდის GitHub-ზე.

`Pull` — GitHub-იდან ცვლილებები მოდის ჩვენთან.

2.4. რა არის Git Fetch?

არსებობს კიდევ ერთი ბრძანება:

```
git fetch origin
```

`git fetch` იღებს დისტანციური რეპოზიტორიის ახალ მონაცემებს, მაგრამ მათ ავტომატურად არ აერთიანებს მიმდინარე ლოკალურ Branch-ში.

მაგალითად, GitHub-ზე დაემატა ახალი `contact.html`.

თუ გამოვიყენებთ:

```
git fetch origin
```

Git მიიღებს ინფორმაციას ახალი commit-ების შესახებ, მაგრამ სამუშაო საქაღალდეში `contact.html` ჯერ არ გამოჩნდება.

შემდეგ შეგვიძლია ვნახოთ, რომელი commit-ებია GitHub-ზე ახალი:

```
git log --oneline main..origin/main
```

ხოლო თუ გვინდა, რომ ცვლილებები ჩვენს ლოკალურ `main` Branch-ში გაერთიანდეს:

```
git pull origin main
```

განსხვავება:

ბრძანება	მოქმედება
`git fetch`	იღებს დისტანციურ მონაცემებს, მაგრამ არ ცვლის მიმდინარე Branch-ის ფაილებს
`git pull`	იღებს მონაცემებს და აერთიანებს ცვლილებებს მიმდინარე Branch-ში

`git pull` ჩვეულებრივ იყენებს Fetch-ს და შემდეგ Merge-ს ან Rebase-ს, Git-ის კონფიგურაციის მიხედვით. ამ გაკვეთილში ვიმუშავებთ Merge-ის მეთოდით.

2.5. რატომ უნდა შევამოწმოთ Git Status Pull-მდე?

სანამ პროექტს განვაახლებთ, სასურველია შევამოწმოთ:

```
git status
```

თუ მივიღებთ:

```
nothing to commit, working tree clean
```

ნიშნავს, რომ სამუშაო საქაღალდეში შეუნახავი ცვლილებები არ გვაქვს.

თუ გვაქვს შეცვლილი ფაილები, რომლებიც ჯერ commit-ში არ არის შენახული, Pull-ის დროს შეიძლება პრობლემა წარმოიშვას.

ამიტომ უსაფრთხო სამუშაო თანმიმდევრობა ასეთია:

```
git status
git checkout main
git pull origin main
```

თუ ცვლილებები გვაქვს, ჯერ უნდა გადავწყვიტოთ, როგორ შევინახოთ ისინი.

3. GitHub-ზე გუნდური პროექტის შექმნა — დეტალური ახსნა

ახლა განვიხილოთ მთავარი პრაქტიკული ნაწილი.

სამი მოსწავლე — ნიკა, მარიამი და ლუკა — აპირებს სკოლის ვებსაიტის შექმნას.

თითოეული იმუშავებს საკუთარ კომპიუტერში, მაგრამ საბოლოოდ ყველაფერი ერთ GitHub რეპოზიტორიაში გაერთიანდება.

3.1. პირველი ნაბიჯი — GitHub-ზე რეპოზიტორიის შექმნა

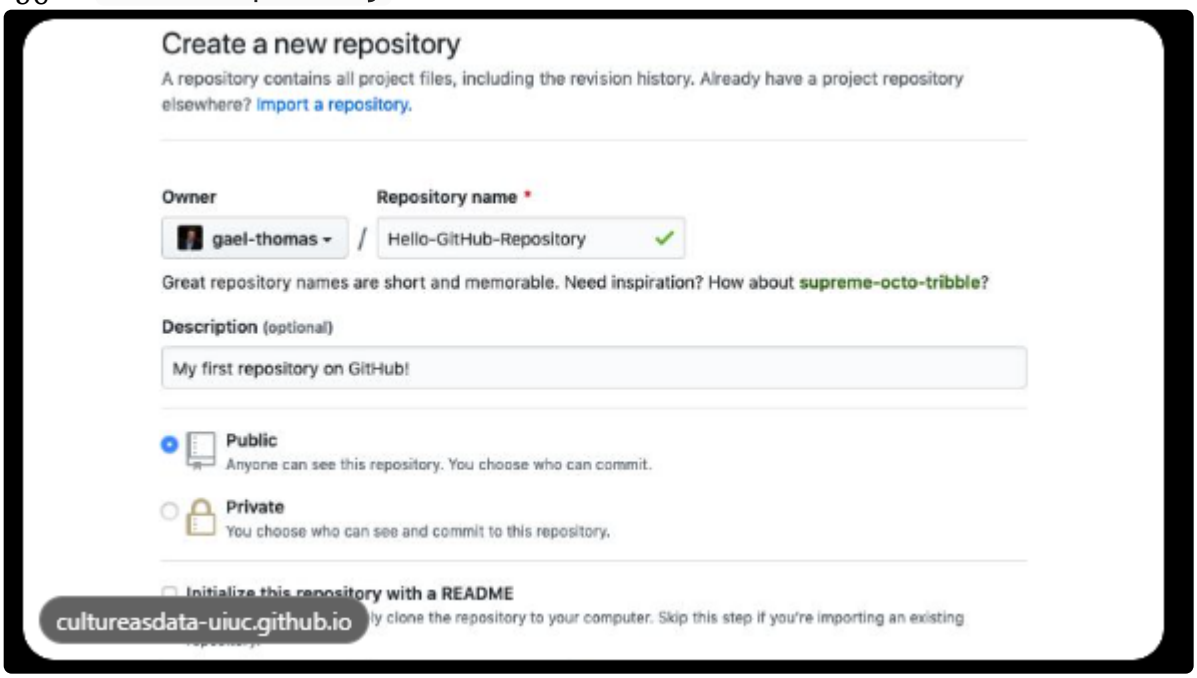
პირველი მოსწავლე, ნიკა, ქმნის საერთო რეპოზიტორიას.

ამისთვის უნდა შევიდეს:

GitHub-ის ოფიციალური ვებსაიტი

შემდეგ ასრულებს მოქმედებებს:

1. შედის საკუთარ GitHub ანგარიშში.
2. ზედა მარჯვენა კუთხეში აჭერს `+` ღილაკს.
3. ირჩევს `New repository`.
4. Repository name ველში წერს `school-website`.
5. ირჩევს `Public` ან `Private`.
6. მონიშნავს `Add a README file`.
7. აჭერს `Create repository`.



ახლა GitHub-ზე შეიქმნა ახალი რეპოზიტორია.

მაგალითად, მისამართი შეიძლება იყოს:

```
https://github.com/nika123/school-website
```

აქ `nika123` არის მაგალითისთვის გამოყენებული Username.

რეალურად მოსწავლეს საკუთარი GitHub Username ექნება.

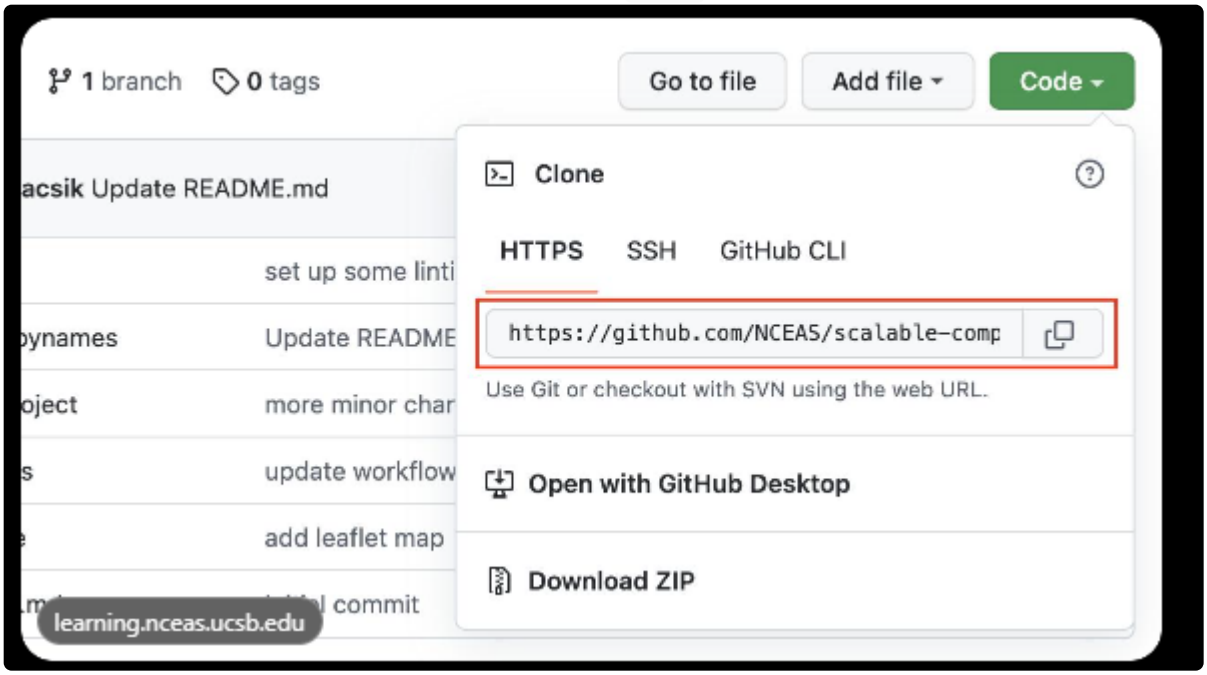
რატომ ვამატებთ README ფაილს?

იმიტომ, რომ ახალი რეპოზიტორია თავიდანვე შეიცავდეს პირველ commit-ს და `main` Branch-ს. ასე მოსწავლეებს გაცილებით მარტივად შეეძლებათ მისი კლონირება და Branch-ების შექმნა.

3.2. მეორე ნაბიჯი — საიდან უნდა დააკოპირონ პროექტის ლინკი?

რეპოზიტორიის შექმნის შემდეგ ნიკა ხედავს პროექტის მთავარ გვერდს.

ფაილების სიის ზემოთ არსებობს ღილაკი `Code`.



ნიკამ უნდა:

1. დააჭიროს `Code` ღილაკს.
2. აირჩიოს `Local` განყოფილება, თუ შესაბამისი ჩანართი ჩანს.
3. აირჩიოს `HTTPS`.
4. დააჭიროს მისამართის გვერდით Copy ღილაკს.

დაკოპირებული ბმული შეიძლება ასე გამოიყურებოდეს:

```
https://github.com/nika123/school-website.git
```

ეს არის Repository URL, რომელსაც მოსწავლეები გამოიყენებენ `git clone` ბრძანებისთვის.

ნიკამ ეს ბმული უნდა გაუგზავნოს მარიამსა და ლუკას.

მაგალითად, Messenger-ში ან კლასის საერთო ჩატში.

GitHub-ის ოფიციალური ინსტრუქციაც აღწერს სწორედ `Code → HTTPS → Copy` მეთოდს.

GitHub Docs

3.3. მესამე ნაბიჯი — სხვა მოსწავლეების დამატება Collaborator-ებად

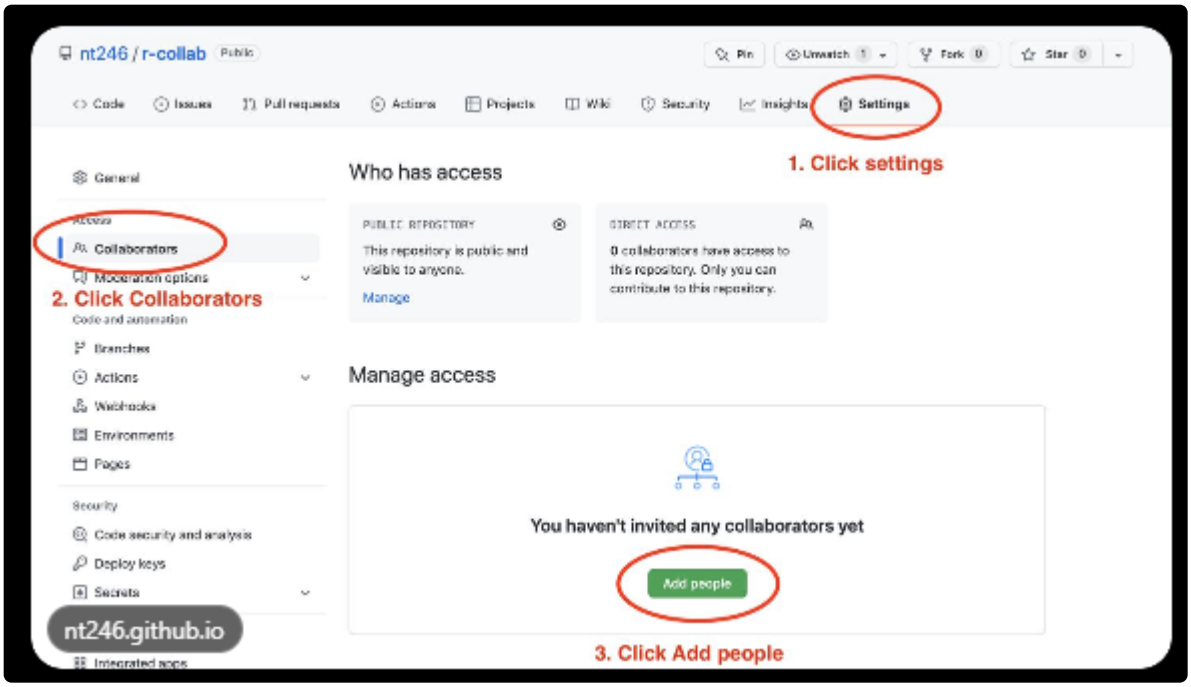
მხოლოდ ბმულის გაგზავნა ყოველთვის საკმარისი არ არის.

თუ რეპოზიტორია საჯაროა, სხვა მოსწავლეს შეუძლია მისი ნახვა და კლონირება, მაგრამ ეს თავისთავად არ აძლევს მას რეპოზიტორიაში ცვლილებების ატვირთვის უფლებას.

თუ გვინდა, რომ მარიამმა და ლუკამ საკუთარი Branch-ები საერთო რეპოზიტორიაში ატვირთონ, ნიკამ ისინი Collaborator-ებად უნდა დაამატოს.

ამისთვის ნიკა:

1. ხსნის თავის `school-website` რეპოზიტორიას.
2. გადადის `Settings` ჩანართში.
3. მარცხენა მენიუში, `Access` განყოფილებაში ირჩევს `Collaborators`-ს.
4. აჭერს `Add people`.
5. საძიებო ველში წერს მარიამის GitHub Username-ს.
6. ირჩევს მის ანგარიშს.
7. აჭერს `Add ... to repository`.
8. იგივე მოქმედებებს იმეორებს ლუკასთვის.



შემდეგ რა ხდება?

მარიამსა და ლუკას მიუვათ მოწვევა GitHub-ზე ან ელფოსტაზე.

მათ უნდა მიიღონ მოწვევა — `Accept invitation`.

მხოლოდ მოწვევის მიღების შემდეგ ექნებათ შესაბამისი წვდომა საერთო რეპოზიტორიაში ცვლილებების გასაგზავნად.

GitHub Docs

3.4. მეოთხე ნაბიჯი — ყველა მოსწავლე აკლონირებს პროექტს

ახლა ნიკამ, მარიამმა და ლუკამ უნდა გადმოიტანონ პროექტი საკუთარ კომპიუტერებში.

ყველა მათგანი VS Code-ში ხსნის Terminal-ს:

`Terminal → New Terminal`

და ასრულებს:

```
git clone https://github.com/nika123/school-website.git
```

შემდეგ:

```
cd school-website
code .
```

ნიკა ამ ბრძანებებს თავის კომპიუტერში ასრულებს.

მარიამი — თავის კომპიუტერში.

ლუკა — თავის კომპიუტერში.

ახლა სამივე მოსწავლეს აქვს ერთი და იმავე პროექტის ლოკალური ასლი.

მნიშვნელოვანია: მხოლოდ ერთმა მოსწავლემ არ უნდა შეასრულოს Clone. ყველა მოსწავლეს პროექტის საკუთარი ლოკალური ასლი უნდა ჰქონდეს.

4. სრული პრაქტიკული მაგალითი — სამი მოსწავლე ერთ პროექტზე

აქ განვიხილავთ მთელ გუნდურ პროცესს, დასაწყისიდან ბოლომდე.

4.1. სამუშაოს განაწილება

სამმა მოსწავლემ უნდა შექმნას სკოლის ვებსაიტი.

მოსწავლე	Branch	სამუშაო
ნიკა	`header`	ნავიგაციის მენიუ
მარიამი	`about-page`	სკოლის შესახებ გვერდი
ლუკა	`contact-page`	საკონტაქტო გვერდი

თითოეული იმუშავებს საკუთარ Branch-ში.

4.2. ნიკა ქმნის პროექტის საწყის ვერსიას

ნიკა, როგორც პროექტის მფლობელი, თავის კომპიუტერში ხსნის კლონირებულ რეპოზიტორიას.

`main` Branch-ში ქმნის ორ ფაილს.

`index.html`:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>School Website</title>
    <link rel="stylesheet" href="style.css">
</head>
<body>
    <h1>Welcome to Our School</h1>
```

```
    <p>This is our team project.</p>
</body>
</html>
```

`style.css`:

```
body {
    font-family: Arial, sans-serif;
    background-color: lightblue;
    text-align: center;
}

h1 {
    color: darkblue;
}
```

ნიკა ინახავს:

```
git add .
git commit -m "Created initial website"
git push origin main
```

ახლა GitHub-ზე პროექტს აქვს საწყისი HTML და CSS ფაილები.

4.3. მარიამი და ლუკა იღებენ საწყის ფაილებს

რადგან მარიამმა და ლუკამ პროექტი მანამდე დაკლონეს, მათი ლოკალური რეპოზიტორიები ჯერ არ შეიცავს ნიკას ახლად დამატებულ HTML/CSS ფაილებს.

მარიამი თავის კომპიუტერში ასრულებს:

```
git checkout main
git pull origin main
```

ლუკაც თავის კომპიუტერში ასრულებს:

```
git checkout main
git pull origin main
```

ახლა სამივე კომპიუტერში არსებობს:

```
school-website/
    README.md
    index.html
    style.css
```

4.4. ნიკა მუშაობს Header-ზე

ნიკა ქმნის ახალ Branch-ს:

```
git checkout -b header
```

`index.html` ფაილში `body` ელემენტის დასაწყისში ამატებს:

```
<header>
    <nav>
        <a href="index.html">Home</a>
        <a href="about.html">About</a>
        <a href="contact.html">Contact</a>
    </nav>
</header>
```

ნიკა ინახავს:

```
git add .
git commit -m "Added navigation menu"
```

და ატვირთავს საკუთარ Branch-ს:

```
git push -u origin header
```

ამის შემდეგ GitHub-ზე არსებობს `header` Branch.

მაგრამ `main` Branch ჯერ არ შეცვლილა.

ეს მნიშვნელოვანია: Branch-ის GitHub-ზე ატვირთვა არ ნიშნავს მის ავტომატურ გაერთიანებას მთავარ Branch-ში.

4.5. მარიამი მუშაობს About გვერდზე

მარიამი თავის კომპიუტერში ქმნის:

```
git checkout -b about-page
```

შემდეგ ახალ ფაილს `about.html`:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>About Our School</title>
</head>
<body>

    <h1>About Our School</h1>

    <p>Our school is located in Khoni.</p>
    <p>We study many interesting subjects.</p>
    <p>We also learn programming.</p>

    <a href="index.html">Back to Home</a>

</body>
</html>
```

შემდეგ:

```
git add .
git commit -m "Created about page"
git push -u origin about-page
```

ახლა GitHub-ზე არსებობს `about-page` Branch-იც.

4.6. ლუკა მუშაობს Contact გვერდზე

ლუკა ქმნის:

```
git checkout -b contact-page
```

შემდეგ ფაილს `contact.html`:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
```

```
    <title>Contact Us</title>
</head>
<body>

    <h1>Contact Our School</h1>

    <p>Email: school@example.com</p>
    <p>Phone: 555 12 34 56</p>

    <form>
        <label for="name">Name:</label>
        <input type="text" id="name">

        <label for="message">Message:</label>
        <textarea id="message"></textarea>

        <button type="submit">Send</button>
    </form>

</body>
</html>
```

შემდეგ:

```
git add .
git commit -m "Created contact page"
git push -u origin contact-page
```

ამ მომენტში GitHub-ზე გვაქვს ოთხი Branch:

```
main
header
about-page
contact-page
```

თითოეულში შესაბამისი ცვლილებებია.

5. როგორ გავაერთიანოთ სამივე მოსწავლის ნამუშევარი?

ახლა გუნდმა უნდა გააერთიანოს ცვლილებები `main` Branch-ში.

ამ მაგალითში გაერთიანებას ნიკა შეასრულებს.

ნაბიჯი 1 — ნიკა იღებს სხვა მოსწავლეების Branch-ების ინფორმაციას

ნიკა თავის კომპიუტერში ასრულებს:

```
git fetch origin
```

ამით Git განაახლებს ინფორმაციას GitHub-ზე არსებული Branch-ებისა და commit-ების შესახებ.

რადგან მარიამმა და ლუკამ Branch-ები თავიანთი კომპიუტერებიდან ატვირთეს, ნიკას ჯერ არ აქვს მათი ლოკალური Branch-ები.

მაგრამ მას შეუძლია გამოიყენოს დისტანციური Branch-ები:

```
origin/about-page
origin/contact-page
```

ნაბიჯი 2 — ნიკა განაახლებს Main Branch-ს

```
git checkout main
git pull origin main
```

ნაბიჯი 3 — ნიკა აერთიანებს Header-ს

```
git merge header
```

ახლა `main` შეიცავს ნავიგაციის მენიუს.

ნაბიჯი 4 — ნიკა აერთიანებს About გვერდს

```
git merge origin/about-page
```

ახლა `main` შეიცავს `about.html`-საც.

ნაბიჯი 5 — ნიკა აერთიანებს Contact გვერდს

```
git merge origin/contact-page
```

ახლა `main` შეიცავს `contact.html`-საც.

რადგან ამ მაგალითში მოსწავლეებმა სხვადასხვა ფაილები შექმნეს, ხოლო ნიკამ Header არსებულ ფაილში დაამატა, ჩვეულებრივ Git ცვლილებებს კონფლიქტის გარეშე გააერთიანებს.

თუ კონფლიქტი მაინც წარმოიშვა, საჭიროა მისი გამოსწორება წინა გაკვეთილის მიხედვით.

ნაბიჯი 6 — ნიკა ატვირთავს განახლებულ Main-ს

```
git push origin main
```

ახლა GitHub-ზე `main` Branch-ში სამივე მოსწავლის ცვლილებები არსებობს.

როგორ გამოიყურება საბოლოო პროექტი?

```
school-website/
    README.md
    index.html
    style.css
    about.html
    contact.html
```

`index.html` ფაილში დამატებულია ნიკას Header.

`about.html` შექმნილია მარიამის მიერ.

`contact.html` შექმნილია ლუკას მიერ.

6. როგორ მიიღონ მარიამმა და ლუკამ საბოლოო ვერსია?

ეს არის გაკვეთილის ყველაზე მნიშვნელოვანი ნაწილი.

ნიკამ უკვე გააერთიანა ყველაფერი და GitHub-ზე ატვირთა.

მაგრამ მარიამისა და ლუკას კომპიუტერებში ჯერ ისევ მათი ძველი ლოკალური ვერსიებია.

მარიამის მოქმედებები

მარიამი თავის VS Code Terminal-ში წერს:

```
git status
```

ამოწმებს, რომ შეუნახავი ცვლილებები არ აქვს.

შემდეგ:

```
git checkout main
git pull origin main
```

ახლა მის კომპიუტერში გამოჩნდება ნიკას Header და ლუკას Contact გვერდიც.

ლუკას მოქმედებები

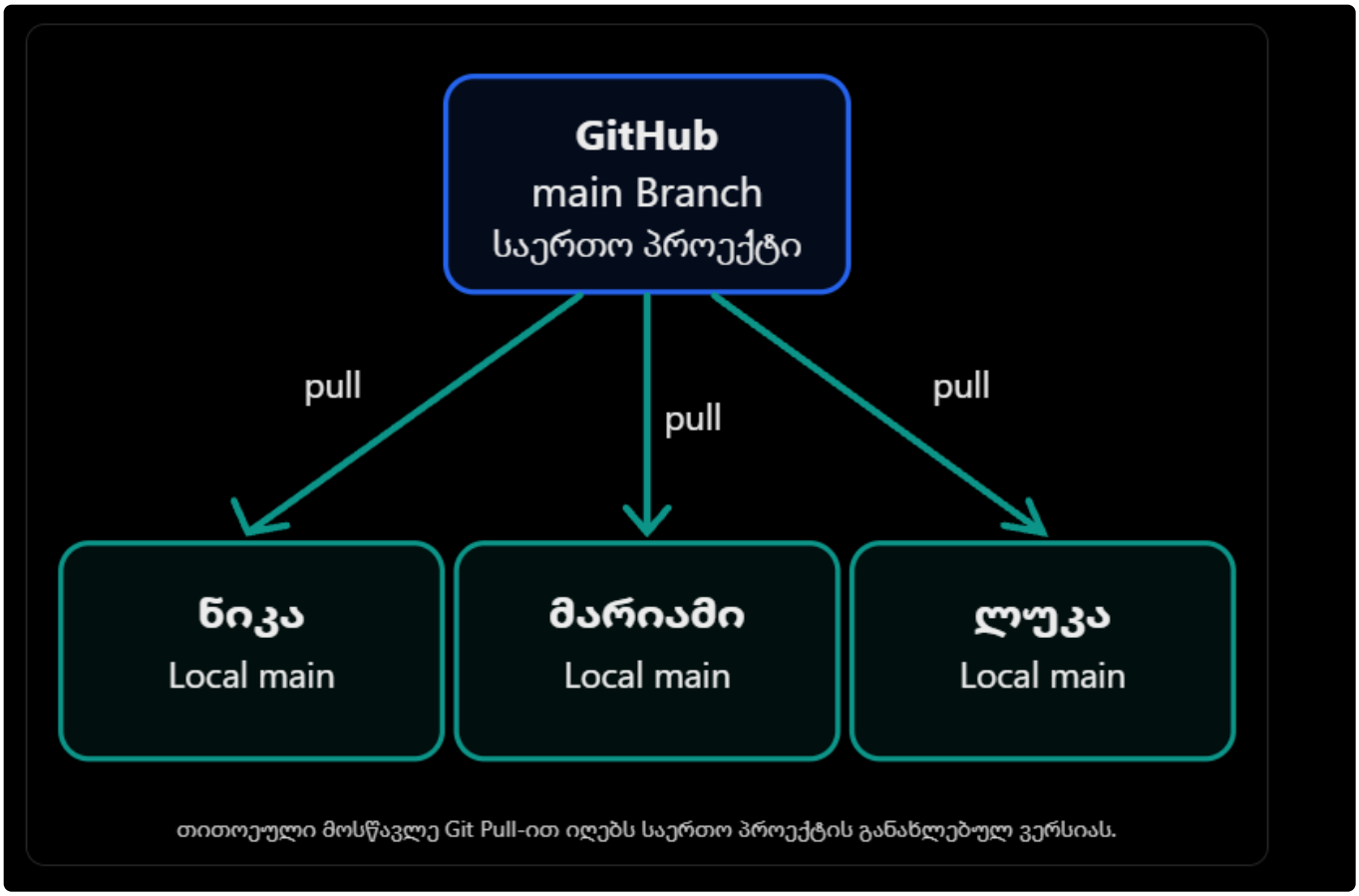
ლუკაც ასრულებს:

```
git status
git checkout main
git pull origin main
```

მის კომპიუტერშიც გამოჩნდება სამივე მოსწავლის ნამუშევარი.

შედეგი: სამივე მოსწავლის ლოკალური `main` Branch განახლებულია და შეიცავს საერთო პროექტის საბოლოო ვერსიას.

თითოეული მოსწავლე Git Pull-ით იღებს საერთო პროექტის განახლებულ ვერსიას.



7. რა მოხდება, თუ ორი მოსწავლე ერთსა და იმავე ფაილს შეცვლის?

განვიხილოთ მეორე სიტუაცია.

ნიკამ `main` Branch-ში შეცვალა:

```
<h1>Welcome to Khoni School</h1>
```

მარიამმა კი თავის Branch-ში იგივე სტრიქონი შეცვალა:

```
<h1>Welcome to Our New School</h1>
```

თუ ამ ორი ცვლილების გაერთიანებას ვცდით, შეიძლება წარმოიშვას Merge Conflict.

Git ასეთ დროს გვაჩვენებს:

```
CONFLICT (content): Merge conflict in index.html
Automatic merge failed;
fix conflicts and then commit the result.
```

ფაილში შეიძლება გამოჩნდეს:

```
<<<<<<< HEAD
<h1>Welcome to Khoni School</h1>
=======
<h1>Welcome to Our New School</h1>
>>>>>>> about-page
```

უნდა ავირჩიოთ საბოლოო ტექსტი.

მაგალითად:

```
<h1>Welcome to Our New Khoni School</h1>
```

შემდეგ წავშალოთ კონფლიქტის ნიშნები და შევინახოთ ფაილი.

```
git add index.html
git commit -m "Resolved merge conflict"
```

ახლა ცვლილებები წარმატებით გაერთიანებულია.

გუნდური მუშაობის მთავარი წესი: უმჯობესია მოსწავლეებმა თავიდანვე გადაანაწილონ სამუშაო ისე, რომ ერთდროულად ერთი და იმავე კოდის შეცვლა ნაკლებად დასჭირდეთ. ეს შეამცირებს კონფლიქტების ალბათობას, თუმცა სრულად ვერ გამორიცხავს მათ.

8. როგორ ვიმუშაოთ ერთ პროექტზე ყოველდღიურად?

როდესაც უკვე ვიცით ყველა ბრძანება, შეგვიძლია ჩამოვაყალიბოთ სტანდარტული სამუშაო პროცესი.

მუშაობის დაწყებამდე

```
git checkout main
git pull origin main
```

ამით ვიწყებთ მთავარი Branch-ის განახლებული ვერსიიდან.

ახალი ფუნქციონალის შექმნა

```
git checkout -b new-feature
```

შემდეგ ვცვლით საჭირო ფაილებს.

ცვლილებების შენახვა

```
git add .
git commit -m "Added new feature"
```

Branch-ის GitHub-ზე ატვირთვა

```
git push -u origin new-feature
```

Branch-ის გაერთიანება

პასუხისმგებელი მოსწავლე ან გუნდის წევრი, რომელსაც შესაბამისი წვდომა აქვს, განაახლებს `main`-ს და გააერთიანებს Branch-ს:

```
git checkout main
git pull origin main
git merge new-feature
git push origin main
```

ეს მაგალითი გულისხმობს, რომ `new-feature` Branch ამ კომპიუტერზე არსებობს და კონფლიქტი არ წარმოიშვა.

გუნდის სხვა წევრების განახლება

```
git checkout main
git pull origin main
```

ასე მუშაობა შეიძლება გაგრძელდეს იმდენჯერ, რამდენჯერაც პროექტში ახალი ფუნქციონალის დამატება დაგვჭირდება.

რეალურ პროექტებში ხშირად იყენებენ Pull Request-ებსაც, რათა ცვლილებები მთავარ Branch-ში გაერთიანებამდე შემოწმდეს.

9. პრაქტიკული სავარჯიშოები

სავარჯიშო 1 — Git Pull-ის გამოყენება (მარტივი)

დავალება: მასწავლებლის მიერ GitHub-ზე დამატებული ცვლილებები მიიღეთ საკუთარ კომპიუტერში.

მოთხოვნები:

1. გახსენით თქვენთვის გაზიარებული სასწავლო პროექტი VS Code-ში.
2. გადადით `main` Branch-ზე.
3. გამოიყენეთ `git status` პროექტის მდგომარეობის შესამოწმებლად.
4. მასწავლებელმა GitHub-ზე დაამატოს ახალი ფაილი სახელით `news.html` და შეინახოს commit-ში.
5. შეასრულეთ `git pull origin main`.
6. შეამოწმეთ, გამოჩნდა თუ არა `news.html` თქვენს პროექტში.
7. გახსენით ფაილი და დააკვირდით მის შიგთავსს.
8. გამოიყენეთ `git log --oneline` და ნახეთ ახალი commit.

შედეგი: მოსწავლის კომპიუტერში უნდა გამოჩნდეს GitHub-ზე დამატებული ახალი ფაილი.

სავარჯიშო 2 — Git Fetch და Git Pull-ის შედარება (საშუალო)

დავალება: შეამოწმეთ, როგორ განსხვავდება `git fetch` და `git pull`.

მოთხოვნები:

1. გახსენით სასწავლო პროექტი და გადადით `main` Branch-ზე.
2. დარწმუნდით, რომ სამუშაო საქაღალდე სუფთაა.
3. მასწავლებელმა GitHub-ზე დაამატოს ახალი ფაილი `events.html` და შექმნას commit.
4. გამოიყენეთ `git fetch origin`.
5. შეამოწმეთ, გამოჩნდა თუ არა ახალი ფაილი სამუშაო საქაღალდეში.
6. გამოიყენეთ `git log --oneline main..origin/main` და დააკვირდით ახალ commit-ს.

7. შემდეგ შეასრულეთ `git pull origin main`.
8. გადაამოწმეთ, გამოჩნდა თუ არა `events.html`.
9. ახსენით, რატომ არ გამოჩნდა ფაილი Fetch-ის შემდეგ, მაგრამ გამოჩნდა Pull-ის შემდეგ.

შედეგი: მოსწავლემ უნდა შეძლოს Fetch-სა და Pull-ს შორის განსხვავების პრაქტიკულად დანახვა.

სავარჯიშო 3 — გუნდური პროექტი (რთული)

დავალება: შექმენით სკოლის ვებსაიტი ორ- ან სამკაციან გუნდში.

მოთხოვნები:

1. ერთმა მოსწავლემ შექმნას GitHub რეპოზიტორია `team-school-website`.
2. Repository-ის შექმნის დროს დაამატოს README.
3. `Settings → Collaborators → Add people` საშუალებით დაამატოს გუნდის სხვა წევრები.
4. მოწვეულმა მოსწავლეებმა მიიღონ მიწვევა.
5. პროექტის მფლობელმა `Code → HTTPS` საშუალებით დააკოპიროს Repository URL და გაუგზავნოს დანარჩენებს.
6. თითოეულმა მოსწავლემ შეასრულოს `git clone`.
7. პროექტის მფლობელმა შექმნას `index.html` და `style.css`, შემდეგ ატვირთოს `main` Branch-ზე.
8. დანარჩენმა წევრებმა მიიღონ საწყისი ფაილები `git pull origin main` ბრძანებით.
9. პირველმა მოსწავლემ შექმნას `header` Branch და დაამატოს ნავიგაციის მენიუ.
10. მეორე მოსწავლემ შექმნას `about-page` Branch და დაამატოს `about.html`.
11. მესამე მოსწავლემ, თუ გუნდში სამი წევრია, შექმნას `contact-page` Branch და დაამატოს `contact.html`.
12. ყველამ შექმნას საკუთარი commit.
13. ყველამ ატვირთოს საკუთარი Branch GitHub-ზე.
14. გუნდის ერთმა წევრმა გამოიყენოს `git fetch origin` და მიიღოს დისტანციური Branch-ების ინფორმაცია.
15. გააერთიანეთ ცვლილებები `main` Branch-ში.
16. ატვირთეთ განახლებული `main` GitHub-ზე.
17. დანარჩენმა წევრებმა შეასრულონ `git pull origin main`.
18. შეამოწმეთ, რომ ყველა მოსწავლეს პროექტის ერთი და იგივე ძირითადი ვერსია აქვს.
19. გამოიყენეთ `git log --oneline --all --graph --decorate` და დაათვალიერეთ ცვლილებების ისტორია.

საბოლოო შედეგი: GitHub-ზე უნდა არსებობდეს საერთო რეპოზიტორია, რომელშიც სამივე მოსწავლის ნამუშევარია გაერთიანებული.